Design and Implementation of a 5-Stage Pipelined RISC-V Processor

Sukeerthi (2024102074)
Padmanjali (2024102047)
Chandralekhya (2024102034)
IIIT Hyderabad

***Abstract*—This report presents the design and implementation of a five-stage pipelined RISC-V processor. The design extends a previously implemented sequential processor by introducing pipelining to improve instruction throughput. The processor executes instructions through five pipeline stages: Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back. Additional hardware such as pipeline registers, forwarding logic, and hazard detection units are implemented to ensure correct operation in the presence of instruction dependencies. Simulation results demonstrate correct functionality and improved execution efficiency compared to the sequential architecture.**

I. Introduction

Modern processors achieve high performance by executing multiple instructions simultaneously using pipelining. In a sequential processor, instructions are executed one after another, meaning that each instruction must complete all stages of execution before the next instruction begins. Although this approach is simple and easy to implement, it results in inefficient utilization of hardware resources.

Pipelining addresses this inefficiency by dividing instruction execution into multiple stages. Each stage performs a portion of the instruction's work, allowing several instructions to be processed concurrently at different stages of execution. As a result, the processor can complete more instructions in the same amount of time.

In this project, a five-stage pipelined RISC-V processor was implemented in Verilog. The design builds upon the sequential processor implemented in the previous phase and modifies the datapath to support pipelined execution. Additional logic was introduced to resolve hazards that arise when instructions overlap in the pipeline.

The pipeline consists of the following stages:

Instruction Fetch (IF)
Instruction Decode (ID)
Execute (EX)
Memory Access (MEM)
Write Back (WB)

Each stage performs a specific function and communicates with adjacent stages through pipeline registers.

II. Sequential Processor Architecture

Before introducing pipelining, a sequential processor architecture was developed. In this design each instruction passes through all execution stages before the next instruction begins execution. Although the sequential architecture guarantees correct instruction execution, it results in significant idle time for many hardware components.

The sequential datapath includes the following major components:

Component	Function
Program Counter	Stores address of current instruction
Instruction Memory	Stores program instructions
Register File	Stores general-purpose registers
ALU	Performs arithmetic and logical operations
Data Memory	Stores program data
Control Unit	Generates control signals

During execution, the processor fetches an instruction from memory, decodes its operands, performs the required computation, accesses memory if necessary, and finally writes the result back to the register file.

III. Instruction Execution Stages

A. Instruction Fetch Stage

The instruction fetch stage is responsible for retrieving the next instruction from instruction memory. The Program Counter (PC) holds the address of the current instruction. During each clock cycle, the PC is used to access instruction memory, which returns the corresponding 32-bit instruction.

Since RISC-V instructions are fixed at 32 bits in length, the PC typically increments by four after every instruction. An adder computes PC+4, which represents the address of the next sequential instruction.

If a branch instruction is taken, the PC is updated with the branch target address instead of PC+4. This allows the processor to alter the control flow of the program.

B. Instruction Decode Stage

In the instruction decode stage, the fetched instruction is interpreted and the required operands are prepared. The instruction fields are separated into opcode, source register addresses, destination register address, and function bits.

The register file reads the values stored in the source registers. At the same time, the immediate generator extracts any constant values embedded within the instruction and sign-extends them so that they can be used by the ALU.

The control unit analyzes the opcode and generates control signals that determine how the instruction will proceed through the remaining stages of the pipeline.

C. Execute Stage

The execute stage performs arithmetic and logical computations using the Arithmetic Logic Unit (ALU). The ALU receives two operands, which may originate either from the register file or from the immediate generator depending on the instruction type.

Typical operations performed in this stage include addition, subtraction, logical operations, comparisons, and address calculations for memory operations.

For branch instructions, the ALU compares the two source operands and determines whether the branch condition is satisfied.

D. Memory Access Stage

The memory access stage performs read and write operations on the data memory. Load instructions read data from memory, while store instructions write data to memory.

If the instruction does not involve memory access, the ALU result simply passes through this stage unchanged.

E. Write Back Stage

The write back stage completes the instruction execution process. The final result of the instruction is written into the destination register in the register file.

The value written back may originate either from the ALU result or from the data memory depending on the instruction type.

IV. Need for Pipelining

In the sequential processor, only one instruction can be processed at a time. This means that the execution of N instructions requires approximately N multiplied by the number of stages in the datapath.

Pipelining improves processor performance by overlapping instruction execution. Instead of waiting for one instruction to finish before starting the next, the processor allows different instructions to occupy different stages simultaneously.

Once the pipeline becomes fully occupied, a new instruction completes during every clock cycle. This significantly increases instruction throughput and allows the processor to utilize its hardware resources more efficiently.

Table I illustrates how instructions move through the pipeline stages over multiple clock cycles.

TABLE I
Example Pipeline Execution Timeline

Cycle	I1	I2	I3	I4	I5
1	IF				
2	ID	IF			
3	EX	ID	IF		
4	MEM	EX	ID	IF	
5	WB	MEM	EX	ID	IF
6		WB	MEM	EX	ID
7			WB	MEM	EX
8				WB	MEM
9					WB

This overlap significantly increases instruction throughput. Although each instruction still requires five stages to complete, multiple instructions are processed simultaneously, allowing the processor to achieve much higher performance compared to a sequential architecture.

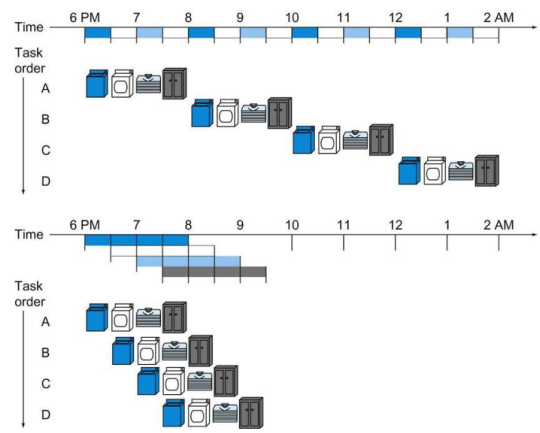

Fig. 1. Laundry analogy

Table II illustrates the difference between sequential and pipelined execution.

TABLE II
Sequential vs Pipelined Execution

Processor Type	Cycles per Instruction
Sequential Processor	5 cycles
Pipelined Processor	$\approx$ 1 cycle (after fill)

Although pipeline hazards and stalls may occasionally introduce delays, the overall performance improvement achieved through pipelining is substantial.

V. Pipeline Architecture

Pipeline registers play an important role in enabling pipelined execution. In a pipelined processor, several instructions are processed at the same time in different stages. Pipeline registers are placed between stages to hold intermediate values so that each stage can work independently during every clock cycle.

These registers capture the outputs produced by one stage at the rising edge of the clock and provide them as inputs to the next stage in the following cycle. This mechanism ensures that instructions move through the pipeline in an organized manner without interfering with each other.

In addition to transferring data between stages, the pipeline registers also support control operations required for hazard handling. For example, they allow instructions to be temporarily held during stalls and can insert NOP operations when a flush is required. This makes it possible to correctly handle data hazards and control hazards while maintaining continuous pipeline execution.

Pipeline Register	Stored Information
IF/ID	Instruction and PC+4
ID/EX	Register operands and immediate values
EX/MEM	ALU result and memory control signals
MEM/WB	Memory data and ALU result

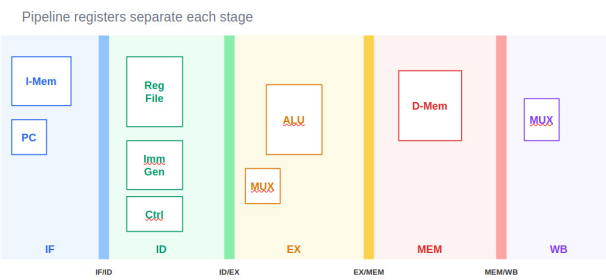

Fig. 2. Registers dividing the datapath into 5 parts

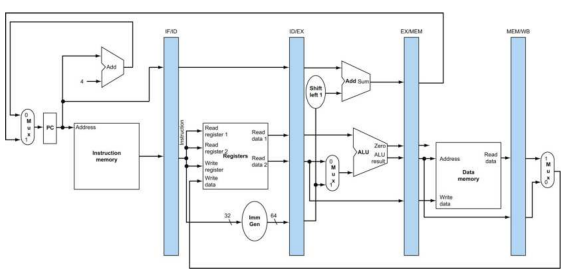

Fig. 3. Five-stage pipelined processor datapath

VI. Propagation of Control Signals

In a pipelined architecture, control signals generated during the instruction decode stage must be propagated through the pipeline so that later stages execute the correct operations.

When an instruction is decoded, the control unit generates signals that determine how the datapath components behave. These signals include register write enable, memory read and write control, ALU operation selection, and the source of data to be written back to the register file.

Since different instructions are present in different pipeline stages simultaneously, each instruction must carry its control signals along with it as it progresses through the pipeline. Pipeline registers are therefore used not only to store data values but also to store control signals.

Table III summarizes some of the key control signals used in the pipelined processor.

TABLE III
Control Signals Used in the Pipeline

Signal	Description
RegWrite	Enables register file write operation
MemRead	Enables memory read operation
MemWrite	Enables memory write operation
MemToReg	Selects memory output for write-back
ALUSrc	Selects ALU operand source
Branch	Indicates branch instruction

By storing these control signals inside pipeline registers, the processor ensures that each instruction receives the correct control inputs as it moves through the pipeline stages.

VII. Structural Hazards

Structural hazards occur when two or more instructions require the same hardware resource at the same time. In a pipelined processor, different instructions occupy different stages simultaneously, and if multiple stages attempt to use the same component during the same clock cycle, a resource conflict can arise.

For example, if a processor uses a single memory for both instructions and data, an instruction in the Instruction Fetch stage may attempt to read the next instruction while another instruction in the Memory stage attempts to access data memory. Since both operations require the same memory hardware, the processor cannot perform both accesses simultaneously.

In such cases, the pipeline must stall until the resource becomes available, which reduces pipeline efficiency. In the design implemented in this project, structural hazards are avoided by using separate instruction and data memories. This allows instruction fetch and data memory access to occur in the same cycle without causing a resource conflict.

VIII. Data Hazards

When instructions are executed in a pipeline, it is possible for one instruction to depend on the result produced by a previous instruction that has not yet completed execution. This situation is referred to as a data hazard.

For example, consider the following instruction sequence:

```
add x1, x2, x3
sub x4, x1, x5
```

The second instruction requires the value of register x1, which is produced by the first instruction. However, the first instruction writes its result to the register file only during the write back stage. The second instruction may reach the execute stage before this result becomes available.

If the processor reads the register file at that time, it will obtain an incorrect value.

IX. Forwarding

To reduce the number of stalls caused by data hazards, a forwarding mechanism is implemented. Forwarding allows results produced by instructions in later stages of the pipeline to be directly routed to the execute stage where they are needed.

Instead of waiting for a value to be written back to the register file, the forwarding unit sends the result directly from intermediate pipeline stages to the ALU input of the dependent instruction. This helps maintain pipeline efficiency by avoiding unnecessary stalls.

The forwarding unit continuously compares the source register numbers of the instruction currently in the Execute (EX) stage with the destination registers of instructions present in later pipeline stages such as EX/MEM and MEM/WB. If a match is detected and the instruction writes to a register, the forwarding logic redirects the most recent value to the ALU input.

The ouputs - two control signals `ForwardA` and `ForwardB` are used as select inputs for two 3-input multiplexers placed before the ALU inputs. Each multiplexer determines the correct operand source for the ALU. The possible inputs to these multiplexers are:

- The original register value read from the register file
- The ALU result from the EX/MEM pipeline stage
- The value written back from the MEM/WB pipeline stage

Based on the forwarding control signals, the multiplexers select the most recent available value for each operand.

For example, if the destination register of an instruction in the EX/MEM stage matches one of the source registers of the instruction currently in the EX stage, the ALU result from the EX/MEM stage is forwarded directly to the ALU input. Similarly, if the destination register in the MEM/WB stage matches the source registers of the current instruction, the value from the write-back stage is forwarded.

After the forwarding logic selects the correct register operand, another multiplexer controlled by the signal `ALUSrc` determines the final second input to the ALU. This multiplexer selects between the forwarded register value and the immediate value generated from the instruction.

By forwarding values from intermediate pipeline stages, the processor ensures that dependent instructions receive the correct data without waiting for the write-back stage, thereby improving pipeline performance.

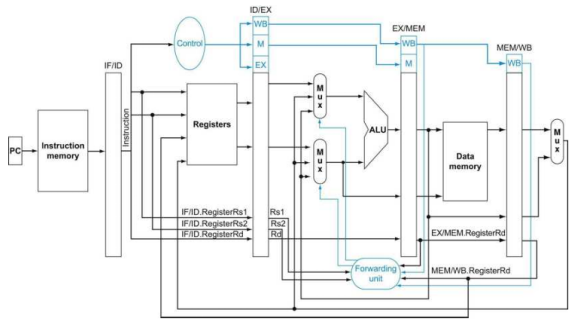

FIGURE 4.54 The datapath modified to resolve hazards via forwarding.

Fig. 4.

X. Load-Use Hazard and Pipeline Stall

Forwarding cannot resolve all data hazards. A special case occurs when a load instruction is followed immediately by an instruction that uses the loaded value.

```
lw x1, 0(x2)
add x3, x1, x4
```

In this case, the value of x1 becomes available only after the memory stage of the load instruction. The next instruction requires this value earlier during its execute stage.

Since forwarding cannot provide the value in time, the pipeline must temporarily pause execution. The hazard detection unit is responsible for identifying situations where forwarding alone cannot resolve data dependencies. In particular, it detects load-use hazards that occur when an instruction attempts to use a value loaded from memory by the immediately preceding instruction.

To resolve this situation, the hazard detection unit monitors the destination register of the instruction currently in the execute stage and compares it with the source registers of the instruction in the decode stage.

If the execute-stage instruction is a load instruction and the destination register matches one of the source registers of the following instruction, the pipeline must stall.

When such a condition is detected, the hazard detection unit generates the following control signals:

Signal	Function
stall_pc	Prevents the program counter from updating
stall_ifid	Freezes the IF/ID pipeline register
flush_idex	Inserts a bubble into the pipeline

These signals ensure that the pipeline temporarily pauses for one cycle, allowing the required data to become available before the dependent instruction proceeds.

XI. Control Hazards

Control hazards are handled using a static branch prediction strategy where the processor initially assumes that a branch will not be taken. Based on this assumption, the pipeline

continues fetching the next sequential instructions while the branch instruction moves through the pipeline.

The actual branch condition is evaluated in the Execute (EX) stage using the ALU. Once the ALU determines the branch outcome, the processor checks whether the earlier assumption was correct. If the branch is indeed taken, the instructions that were fetched assuming sequential execution are incorrect.

These incorrectly fetched instructions are removed by flushing the pipeline registers, effectively replacing them with NOP operations. Since the branch decision is resolved in the EX stage, two subsequent instructions may already be present in the pipeline. As a result, when a branch is taken, these two instructions must be flushed, leading to a two-cycle branch penalty.

1) Sequential Program Counter Update: Under normal operation, the program counter increments by 4 in order to fetch the next instruction. This value represents the address of the next sequential instruction and is used as the default next program counter.

2) Branch Target Calculation: For branch instructions, the processor calculates the target address by adding the sign-extended immediate value to the program counter. The branch target is computed in the execute stage. If the branch condition is satisfied, execution jumps to this calculated target address.

3) Branch Decision: The branch decision depends on both the branch control signal and the comparison result generated by the ALU. A branch is taken only when the branch signal is active and the condition evaluates to true.

4) Pipeline Flush: Since the processor assumes branches are not taken, the next sequential instruction may already be in the pipeline when the branch decision is made. If the branch is taken, this instruction must be discarded.

This is done by flushing the instruction in the pipeline and replacing it with a NOP instruction.

```
assign flush_ifid = branch_taken;
```

When this signal is asserted, the incorrectly fetched instruction in the IF/ID pipeline register is replaced with a no-operation instruction, preventing incorrect execution.

XII. Simulation Results

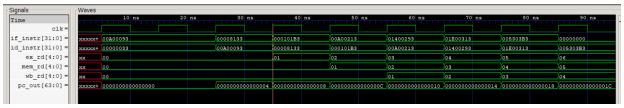

Fig. 5. Waveform showing instruction progression

The waveform shows correct pipeline execution. Instructions fetched in the IF stage move sequentially through ID, EX, MEM, and WB stages every clock cycle. The destination registers propagate through pipeline registers (ex-rd, mem-rd, wb-rd) confirming correct pipeline progression without stalls.

A. Forwarding Verification

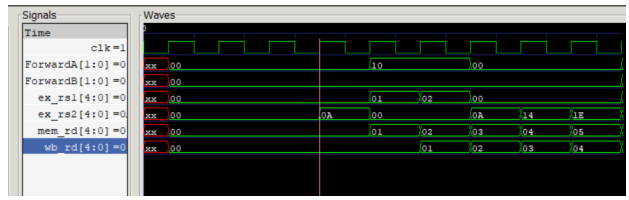

Fig. 6. Waveform demonstrating data forwarding

When the source register in the EX stage (ex-rs1 = 01) becomes the same as the destination register in the MEM stage (mem-rd = 01), ForwardA changes to 10, indicating that the required value is forwarded from the MEM stage to the ALU. After this dependency is resolved, ForwardA returns to 00, while ForwardB remains 00 throughout since no similar match occurs for ex-rs2. Meanwhile, mem-rd and wb-rd continue changing sequentially, showing normal pipeline progression.

```
addi x1, x0, 10
add  x2, x1, x0
add  x3, x2, x0
addi x4, x0, 10
addi x5, x0, 20
addi x6, x0, 30
add  x7, x6, x5
```

B. Pipeline Stall Verification

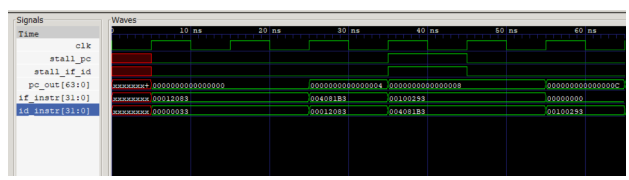

Fig. 7. Waveform demonstrating pipeline stall

The processor first fetches the instruction lw x1,0(x2) and then the next instruction add x3,x1,x4. Since the add instruction needs the value of x1 that is still being loaded by the lw instruction, the processor briefly pauses the pipeline. During this pause, the signals stall-pc and stall-if-id become 1, the program counter stops increasing, and the same instruction stays in the pipeline for one extra cycle. After this one-cycle delay, the pipeline continues normally and the next instruction (addi x5,x0,1) moves forward.

```
lw   x1, 0(x2)
add  x3, x1, x4
addi x5, x0, 1
```

C. Branch Flush Verification

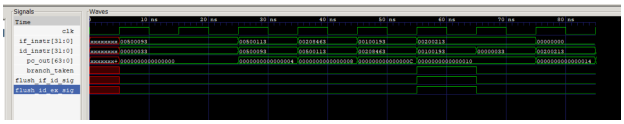

Fig. 8. Waveform demonstrating pipeline flush

Initially, instructions move normally through the IF and ID stages while the program counter (pc-out) increases sequentially. When the branch instruction (00208463) is evaluated, the signal branch-taken becomes 1, indicating the branch condition is true. As a result, the control signals flush-if-id-sig and flush-id-ex-sig go high for one cycle, clearing the incorrect instructions that were fetched after the branch. A NOP (00000033) appears in the pipeline, showing that those instructions were flushed. After this cycle, the pipeline continues execution from the correct branch target.

```
addi x1,x0,5
addi x2,x0,5
beq  x1,x2,8
addi x3,x0,1
addi x4,x0,2
```

D. Pipeline Execution Example

The following RISC-V assembly program was executed on the pipelined processor.

```
addi x1, x0, 10
add  x2, x1, x0
add  x3, x2, x0
addi x4, x0, 10
addi x5, x0, 20
addi x6, x0, 30
add  x7, x6, x5
```

After execution, the register file produced the following output :

```
x0  = 0000000000000000
x1  = 000000000000000a
x2  = 000000000000000a
x3  = 000000000000000a

Total Cycles = 8
```

1) Register Updates: Registers are only updated during the Write Back (WB) stage. From the pipeline schedule:

- Instruction 1 writes to $x1$ in cycle 5
- Instruction 2 writes to $x2$ in cycle 6
- Instruction 3 writes to $x3$ in cycle 7

The simulation stops at cycle 8 since the PC reaches the EOF of the instruction memory, therefore only the first three instructions complete the WB stage. As a result, only registers $x1$, $x2$, and $x3$ are updated.

The remaining instructions are still in earlier stages of the pipeline and have not yet reached the WB stage, which is why registers $x4$ to $x7$ remain unchanged.

E. Verification

```
addi x2, x0, 5
addi x3, x0, 10
add x1, x2, x3
sub x2, x2, x3
and x4, x3, x3
and x5, x3, x4
or x6, x2, x4
or x7, x2, x3
sd x1, 0(x5)
ld x10, 0(x5)
sd x6, 24(x5)
ld x11, 24(x5)
beq x4, x5, 8
beq x0, x0, 0
add x13, x1, x10
add x0, x0, x0
add x0, x0, x0
add x0, x0, x0
add x0, x0, x0
```

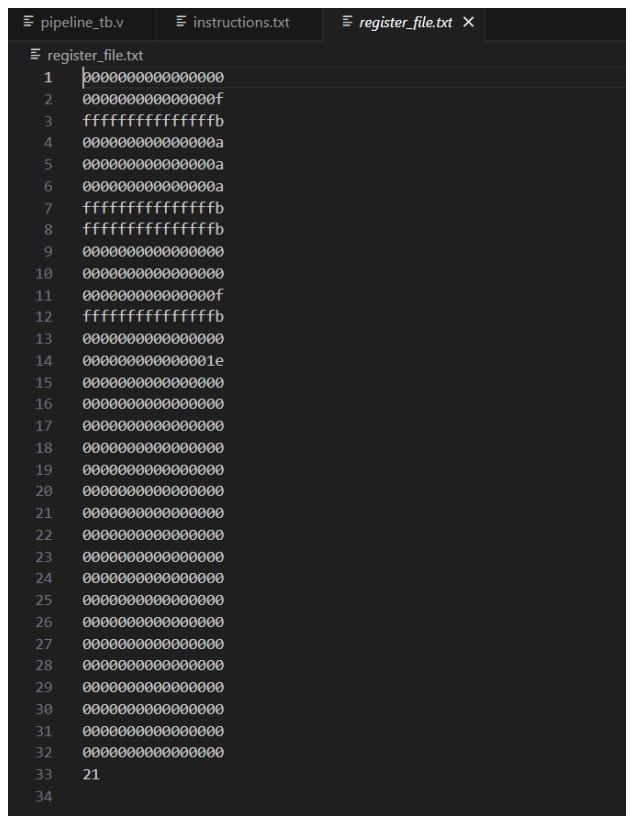

Fig. 9. registers-file output for given instructions

The output matches the expected register-file.txt contents

XIII. Conclusion

This project successfully implements a five-stage pipelined RISC-V processor. By dividing instruction execution into multiple stages and allowing instructions to overlap in execution, the pipeline significantly improves instruction throughput compared to the sequential design.

Hazard handling mechanisms such as forwarding, pipeline stalling, and branch flushing ensure correct execution in the presence of instruction dependencies.

Simulation results confirm that the processor operates correctly under various instruction sequences.

Team contribution

Sukeerthi implemented the pipeline registers for all stages of the processor and assisted in integrating the datapath across the pipeline and helped with the report.

Padmanjali designed and implemented the forwarding unit, contributed to integrating and testing datapath modules across the pipeline, and verified correct data hazard resolution through waveform analysis.

Chandralekhya implemented the hazard detection unit and control hazard handling, including stall and flush logic, and prepared and organized the final project report.